Firmware Block Implementation for the Proto-DUNEII Hit Finding Architecture with the HLS Tool - First Semester Report

Joshua Horswill

January 27, 2021

Abstract

The Deep Underground Neutrino Experiment (DUNE) is a large-scale international project being developed at CERN and will operate within North America. It will consist of two underground neutrino detectors placed within an intense neutrino beam fired from Fermilab, the furthest of which will be 1300 kilometres from the source, in the Sanford Underground Research Facility (SURF). The main objectives of the experiment include answering fundamental questions about the origin of matter, namely certain properties of leptonic CP violation. There are also aims to investigate the unification of forces through the analysis of proton decay, and the formation of black holes through supernova neutrino detection[1]. This report focusses on the TPG cores within the front-end electronics of the detectors, specifically the hit finder architecture that processes optical data generated from argon scintillation within the time projection chambers, as a result of neutrino-nucleon interactions. The focus of discussion will be on the replacement of existing firmware algorithms written for the detector's Field-Programmable Gate Arrays (FPGAs) with designs synthesised via Vivado High Level Synthesis (HLS) from C++ code. The purpose of this is to test the efficiency and convenience of the block design process when compared to writing firmware from scratch in VHDL.

Contents

1 Introduction

1.1 DUNE Experiment Background and Goals

The DUNE experiment is an amalgamation of many neutrino-physics communities, centred around an opportunity provided by the large investment of the U.S. Department of Energy. This will contribute to a large expansion of the underground experimental architecture of the SURF facility, as well as a megawatt neutrino-beam facility 1300km across North America at Fermilab. This investigation will be groundbreaking for neutrino oscillation studies, searches for proton decay, and for neutrino based astrophysics (black hole and supernova burst phenomena).

Analysing the properties of neutrinos has led to evidence for physics beyond the Standard Model (SM). Oscillations of neutrino flavour i.e. a neutrino's ability to transform their flavour after moving through space, is a well established concept. Conclusions can be made from this, including the fact that neutrinos have mass, although unexpectedly small. The sum of the three neutrino mass eigenstates must be less than one million of an electron mass eigenstate [2, 3]. The origin of these small neutrino masses and their oscillations is still not confirmed however, and is up for debate. Finding the explanation will require more precision and detail in the available experimental information. DUNE exists to carry out an investigation of neutrino oscillations to test 'Lepton sector CP Violation' (LCPV) as well as the ordering of the neutrino masses and the three-neutrino paradigm, in which the quantum mechanical mixing of the three neutrino mass eigenstates produces the three known neutrino flavour states [4].

CPV is well established in Quantum Chromodynamic (QCD), allowed by the complex-natured Cabibbo-Kobayashi-Mashawa (CKM) matrix[1][5]. Therefore one would expect to discover an entirely analagous mechanism to arise in the lepton sector of the standard model (SM), leading to LCPV. One of the goals of this collaboration is to determine the rates at which neutrino CP violation occurs within the lepton sector. Another hypotheses proposes that there is a connection between low-energy neutrino physics and leptogenesis [6], which could provide insight into the scenario that led to the baryon asymmetry of the universe. These questions can only by answered by constructing suitable detector equipment; the next section will explore the features of ProtoDUNE's detector's front-end electronics and Data Acquisition (DAQ) equipment. This will feed into the following section that discusses the HLS firmware blocks that could instruct the front-end FPGA-based data processors.

1.2 LAr TPC and Front-End Detector Electronics Overview

The use of Time Projection Chamber modules of Liquid Argon (LAr TPCs) is a matured and popular technique used by several short-baseline[2] neutrino

[1] Used to contain the information on the strength of the flavour-changing weak interaction [5].

[2] The baseline of a neutrino experiment refers to the scale of the beam travel distance. DUNE is 'long baseline' as the detector is positioned across the continent from the beam

experiments. They are popular for several reasons:

- Argon is a noble element which has vanishing electronegativity[3] which means that electrons produced by ionizing radiation will not be absorbed as they drift towards the detector.
- Liquid Argon is inexpensive yet very dense, increasing the likelihood of a particle interacting by 1000 times when compared to Nygren's TPC design [7]. This is important because the cross section for neutrino-nucleon interactions is very small.
- A neutrino-nucleon interaction causes the Argon to scintillate; a secondary energetic charged particle is generated and travels through the liquid, which causes the release of scintillation photons, the number of which is proportional to the energy deposited by the passing particle [8]. This can contribute to a reconstruction of the event.

There exists both single and dual-phase versions of these LAr TPC modules. Inside single-phase liquid argon detectors, such as the structure featured in figure 1, the interactions are readout by wires immersed in the liquid and do not require the amplification of the ionized electron charge. The charge is localised using AC induction and collection wire planes with different orientations. This was tested by ProtonDune-SP at CERN.

The dual phase detectors rely on the amplification of ionisation charges in the ultra-pure cold-argon vapour layer above the liquid, obtaining low-energy detection thresholds with high signal-to-noise ratio over drift distances exceeding 10m. In dual-phase LAr TPCs, ionisation charges produced by neutrino interactions drift vertically towards the liquid-vapour boundary where they are extracted into the 'gas phase', amplified by components named Large Electron Multipliers (LEMs), and collected by segmented anodes; the electron extraction, amplification and collection are performed inside a 3×1 m^2 frame called the charge readout plane (CRP). Bear in mind the dimensions of the detector prototype are $3 \times 1 \times 1$ m^3.

The LAr TPC modules used in the ProtoDUNE II design contain 150 anode plane assemblies (APA's) with 2560 wires per APA. There are 10 fibres/data streams in the APA, each with 256 wires - hence the total of 2560. The Front End LInk eXchange system is used to receive the data from the APAs, and was first developed within the ATLAS collaboration. It is based on custom FPGA-based PCIe I/O cards, combined with commodity servers [10]. Within this system contains the focus of this project: the Trigger Primitive Generation (TPG) core, which will be discussed in more detail later.

The readout window per event is between 3-5.5 ms for a 2.25 ms drift time. The FELIX architecture that handles this influx of events consists of 10 MGT modules, whose function is to process the APA data stream into computational

source.

[3]Low tendency to attract a shared pair of electrons in an atomic bond.

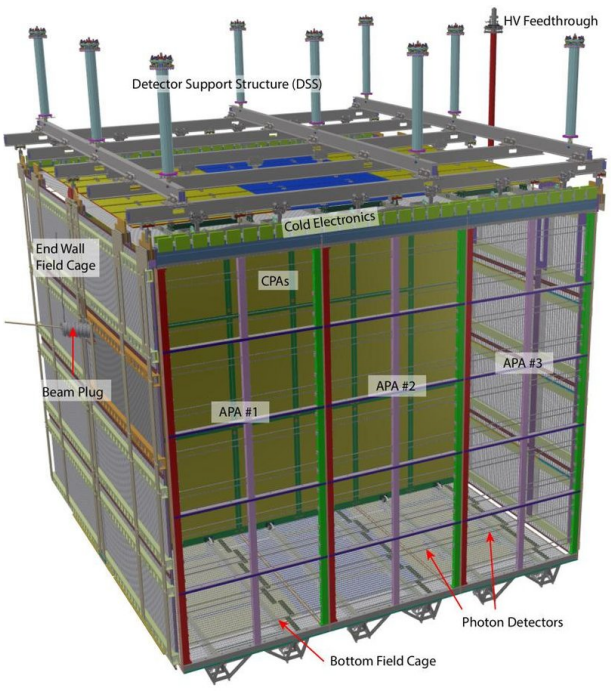

Figure 1: Diagram of the ProtoDUNE single-phase LAr TPC module. This contains two drift volumes, separated by a central cathode plane that is flanked by two anode planes, connected to a field cage that surrounds the entire module. Each anode plane is made of three Anode Plane Assemblies (APAs). These connect and send data to the front-end electronics being discussed in this report [9].

bits, that feed data into 10 Hit Finders (HF). The hits are produced by processing the incoming ADC (Analog-to-Digital Converted) samples in parallel. The MGT output enters the HFs via the data router[4] and is then transferred into 10 sets of 4 TPG block processors; one set for each MGT output. These are multiplexed[5] by the arbitrator into the Central Router (CR) interface [11, 12]. See figure 3 for a visualisation of the packet's journey from the MGT output to the CR, and figure 4 for the dataflow overview.

1.3 The TPG Core

Each TPG unit processes the data from 64 APA wires, and so there are 4 of them for each core[6]. The TPG data consists of a header and payload. The header determines the source wire, the time stamp, and the origin of the data. The payload is 64 bits of ADC sample data for hit finding. There is an AXI4-stream protocol for unpacking and reordering of data in the TPG block, featuring 4 general signals for various instructions in each component:

1. tvalid denotes if the bus data is valid for processing
2. tuser is a flag that denotes the end of a packet frame
3. tlast indicates the end of a packet
4. tready is used to apply back-pressure if the processing units ahead cannot process the data fast enough (all throughput is frozen).

Within the TPG core is a mechanism that strips the packet header; the Header Stripper/Combiner (HSC), featured in figure 2. The header is sent to the output of the block. Meanwhile, the remaining 64 ADC samples and their associated AXI4 signals are processed by the PedSub (pedestal subtraction), FIR (finite impulse response), and HF sub-blocks in the chain. The data router is then signalled to send the next packet.

1.4 System Architecture

We have to consider some factors when studying the DUNE DAQ (data acquisition) system:

1. Hardware and maintenance cost per channel.
2. Technology life-cycle (TLC) of the components. We need long-lasting essential components such as the FPGA (Field Programmable Gate Array) and the network components.

[4]The data router receives multiple-channel, single clock-tick packets. The packets initially exist in a format containing the first sample for all channels, and when processed by the data router are converted into a format subsisting of all the samples for a given channel i.e. multiple clock ticks of data.

[5]Combined into a singular signal output

[6]$4 \times 64 = 256$, the number of wires in a fiber, ten fibers for each MGT accounts for all wires within the APA

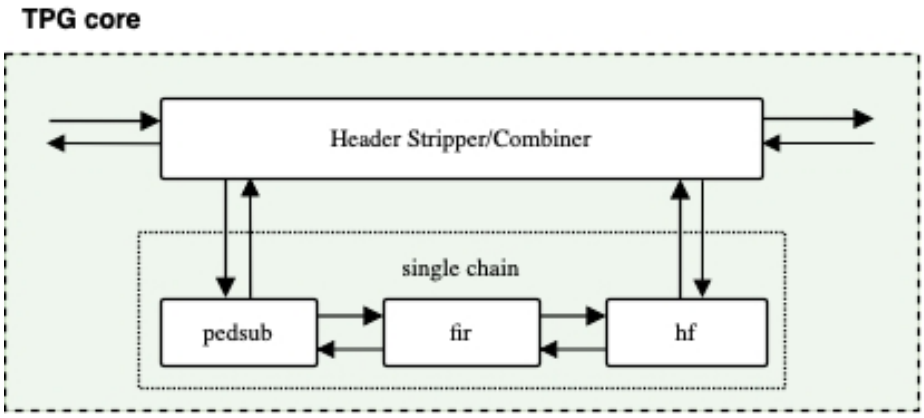

Figure 2: Focus on the header stripper/combiner and the sub-block chain [13].

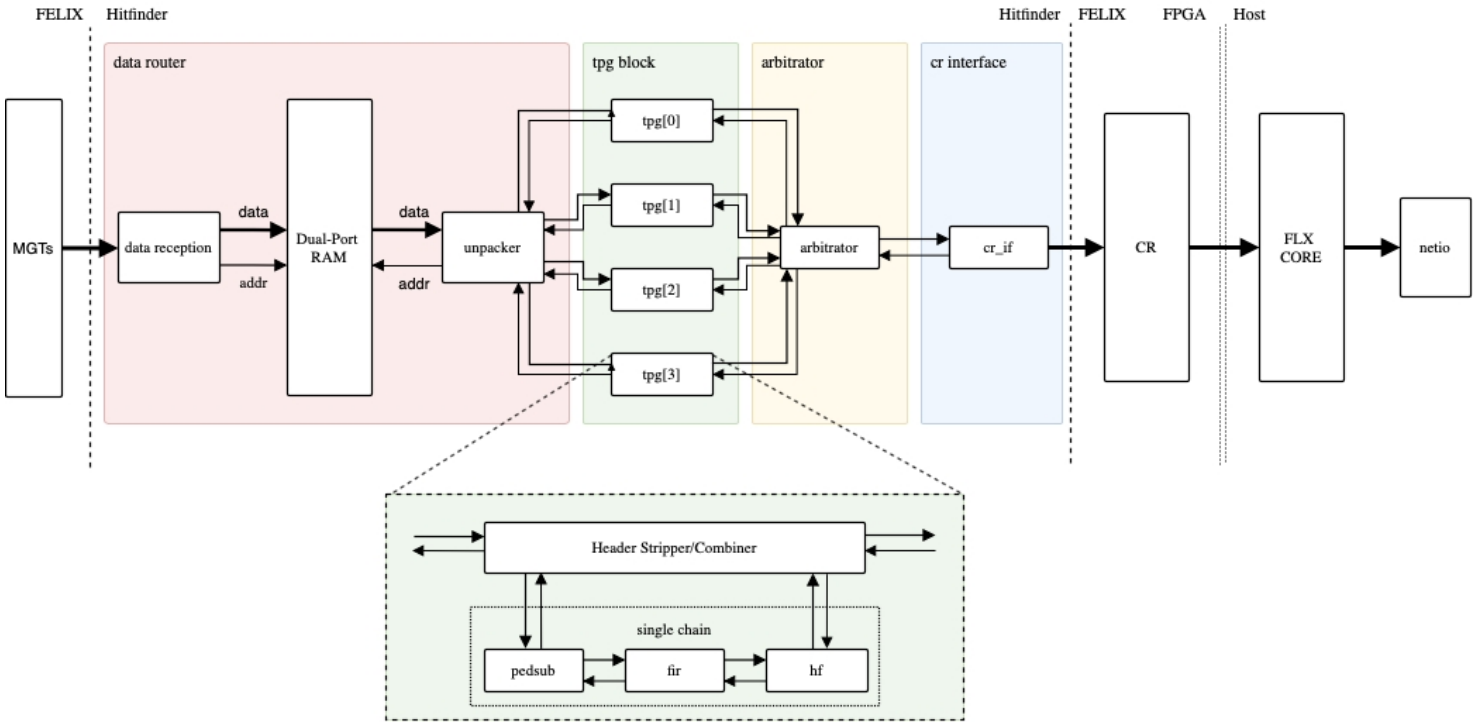

Figure 3: Hit Finder Firmware Architecture [13].

3. Component power consumption: deep underground components with higher power consumption will increase the cooling and running cost.

4. DUNE will run for 20 years so the reliability and maintenance of components are more important than in other large scale projects.

5. Emphasis on modular design of hardware and firmware to avoid a change affecting the rest of the sub-modules.

6. Fast R&D and fast firmware modification: If the tools used are more abstract and are rooted in C++, this will appeal to the vast majority of the users of this tech (physicists).

Requirements for the architectural design of DUNE DAQ:

1. Enough bandwidth provided for the data stream

2. Supernova detection, trigger buffer

3. Detector performance, electronics' noise, level and source (?)

There are strong trade offs between the above requirements:

- High bandwidth networking is expensive, but could be avoided using trigger primitives (?), but the efficiency of this trigger depends on the detector noise performance.

- One can reduce the noise in the acquired signal by applying a digital filter in the front end DAQ before the trigger generating algorithm. However, this is at the cost of using lots of gates and resources in the FPGA.

DAQ architecture can be split into two parts:

- Front end DAQ, comprised of electronics that acquire the data stream from readout-electronics from the TPC detector, so it can process and deliver it to the back end (computation farm).

- Back-end DAQ, comprised of an offline storing, reconstruction computation farm.

2 FELIX

FELIX is a PCIe based high-throughput approach for interfacing front-end and trigger electronics in the ATLAS Upgrade framework:

- "The ATLAS Phase-I upgrade (2018) requires a Trigger and Data Acquisition (TDAQ) system able to trigger and record data from up to three times the nominal LHC instantaneous luminosity."

- The FELIX system provides the infrastructure to achieve this in a scalable, detector agnostic and easily upgradeable way.
- FELIX enables the reduction of custom electronics in favour of software running on commercial servers.
- FELIX is a PC-based networking gateway with the FPGA mounted on PCIe Gen3 cards (high speed connections used in motherboards to connect GPUs, SSD's, wifi chips e.t.c)

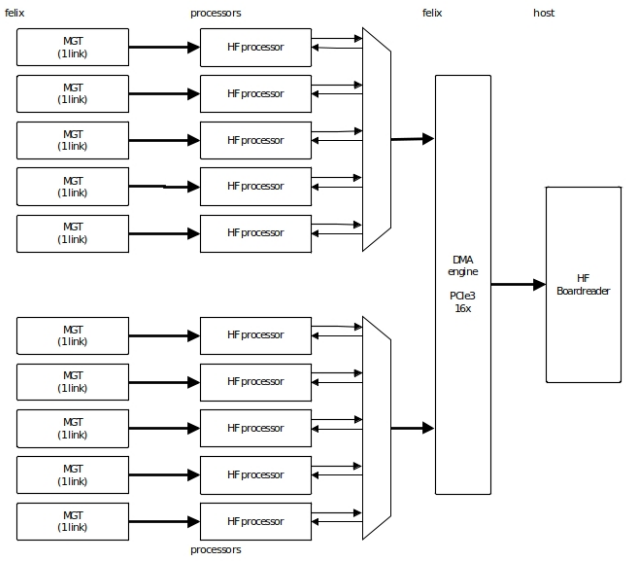

Figure 4: Overview of the Felix architecture in ProtoDUNE [13]

2.1 Data transfer

On one side of the FELIX system, the GigaBit Transceiver (GBT) architecture and a protocol developed by CERN provides 4.8 Gb/s radiation-hard optical link for data transmission from the on-detector front-end electronics. When all the data is multiplexed, the GBT protocol provides up to 42 independent data links, sharing the same fibre. The GBT frame consists of a 116 bit data (payload) and a 4 bit header, the latter of which is used to align the frame at the receiver side. It can be DATA (0101) or IDLE (0110).

FELIX has to handle the Time, Trigger and Control (TTC) system [7], that is based on the recovered LHC clock, by forwarding the machine-synchronous trigger information. This info will be distributed to the aforementioned on-detector

[7] a system that executes one or more sets of tasks according to a pre-determined and set task schedule

electronics over low and fixed latency GBT links. It will also be distributed to the first-level trigger systems.

For the FrontEnds which needs a higher throughput toward FELIX, a customized lightweight FULL mode is defined. The link speed is 9.6 Gb/s, but as data is encoded in 8b/10b a maximum user payload of 7.68 Gb/s can be achieved, which is an improvement on the standard 4.8 Gb/s [14].

3 4 Tonne Demonstrator for Large-Scale Dual-Phase Liquid Argon TPC's

10 kilo-tonne dual-phase liquid argon TPC is one of the main detector options considered for DUNE. The detector relies on amplification of the ionisation charge in ultra-pure argon vapour, which holds several advantages over the single-phase liquid argon TPC. LAr TPC's offer unprecedented 3D imaging capabilities and the functionality of a homogeneous calorimeter. High-resolution imaging is the key to efficient background noise rejection.

When a powerful neutrino beam is hypothetically shot from the United States' Fermilab to these LAr TPC's in the Sanford Underground Research facility in South Dakota, DUNE aims to search for the leptonic CP violation and determine the ordering of the neutrino masses.

Another objective is to extend the sensitivity of proton lifetime in a variety of possible decay channels, and collect high-statistics neutrino samples from atmospheric and astronomical sources.

3.1 Detector details

Single phase liquid argon detectors, readout by wires immersed in the liquid, do not require the amplification of the ionized electron charge. The charge is localised using alternative induction and collection wire planes with different orientations. This was tested by ProtonDune-SP at CERN.

The dual phase version is between 20 and 50 ktonnes and, in contrast to the previous example, relies on the amplification of ionisation charges in the ultra-pure cold argon vapour layer above the liquid, obtaining low-energy detection thresholds with high signal-to-noise ratio over drift distances exceeding 10m.

"Ionisation charges produced by neutrino interactions would drift vertically towards the liquid-vapour boundary where they are extracted into the gas phase, amplified by Large Electron Multipliers (LEMs), and collected on finely segmented anodes"; the electron extraction, amplification and collection are performed inside a 3 x 1 m^2 frame called the charge readout plane (CRP). Bear in mind the detector prototype volume is 3 x 1 x 1 m^3.

Five photo-multiplier tubes are mounted underneath the TPC drift cage. They detect the Argon (prompt S1) scintillation photons formed from passing charged particles, as well as the (proportional S2) secondary scintillation light from the electroluminescence of the electrons extracted in the argon vapour. S1

is at least several orders of magnitude higher in detection amplitude than S2 in a tested TPC.

A wavelength shifter converts the deep ultraviolet photons to the visible spectrum within the sensitivity of the receiving PMT photo-cathode. These photo-cathodes give the absolute time on an event T_0. The CRP applies an electric field in the liquid above 2 kV/cm to move the electrons into the vapour.

There was a measured correlation between the electroluminescence in the gas from S2 scintillation, and the collected charge from the CRP system, as you would intuitively expect [15].

4 HLS

High-level synthesis (HLS) is an automated design process that interprets an algorithmic description of a desired behavior and creates digital hardware that implements that behavior.

Synthesis begins with a high-level specification of the problem, where behavior is generally decoupled from low-level circuit mechanics such as clock-level timing. Early HLS explored a variety of input specification languages, although recent research and commercial applications generally accept synthesizable subsets of ANSI C/C++/SystemC/MATLAB.

The code is analysed, architecturally constrained, and scheduled to transcompile into a register-transfer level (RTL) design in a hardware description language (HDL), which is in turn commonly synthesized to the gate level by the use of a logic synthesis tool.

RTL is a design abstraction which models a synchronous[8] digital circuit in terms of the flow of data between hardware registers[9], and the logical operations performed on those signals.

RTL is used in HDLs i.e. languages that describe the structure and behaviour of electronic circuits, and more commonly digital logic circuits. It allows for circuitry simulation and digital analysis. HDLs look like programming languages such as C, but the main difference is that HDLs include the notion of time. RTLs are used in HDLs to create high-level representations of a circuit, from which lower-level representations and ultimately actual wiring can be derived. Design at the RTL levle is typical practise in modern digital design.

RTL clock cycle data is given in Vivado performance estimate data, as it can read how long a certain block, loop or function in the code took to complete; this is called the latency.

[8] A circuit that changes the state of memory elements in a way synchronised by a clock signal

[9] Circuits composed of flip flops - a sub circuit that has two stable states used to store state information (a basic storage element in sequential logic). These hardware registers have the ability to read or write multiple bits at a time and can use an address to select a particular register. They are used as an interface between the software and the computer peripherals.

4.1 HLS Design Flow

1. Compile, execute (simulate), and debug the C algorithm.

   - This involves validating that the C program correctly implements the required processes.

   There is also post-synthesis validation that compares the synthesised RTL and C programs to see if they perform the same actions.

   In order for the former to occur, there must be a test bench C script that includes a top-level function main() as well as the function to be synthesised.

2. Synthesize the C algorithm into an RTL implementation, optionally using user optimization directives.

3. Generate comprehensive reports and analyze the design.

   After C synthesis, output files are stored in a directory named `syn` which contain folders for each RTL output format, and a report folder. The report folder contains files reporting on the performance and design corresponding to the top-level function and each individual subfunction.

4. Verify the RTL implementation using a pushbutton flow.

5. Package the RTL implementation into a selection of IP formats.

4.1.1 Inputs

- C function written in C, C++, or SystemC

  This is the primary input to Vivado HLS.

- Constraints

  Constraints are required and include the clock period, clock uncertainty, and FPGA target. The clock uncertainty defaults to 12.5% of the clock period if not specified.

- Directives

  Directives are optional and direct the synthesis process to implement a specific behavior or optimization.

- C test bench and associated files

  Vivado HLS uses the C test bench to simulate the C function prior to synthesis and to verify the RTL output using C/RTL co-simulation.

4.1.2 Outputs

- RTL implementation files in hardware description language (HDL) formats

  This is the primary output from Vivado HLS. Using Vivado synthesis, you can synthesize the RTL into a gate-level implementation and an FPGA bitstream file.

  Vivado HLS packages the implementation files as an IP block for use with other tools in the Xilinx design flow. Using logic synthesis, you can synthesize the packaged IP into an FPGA bitstream.

- Report files

  This output is the result of synthesis, C/RTL co-simulation, and IP packaging.

4.2 Optimization

Different optimization directives can be applied before C synthesis:

- Instruct a code task to execute in a pipeline, allowing the next execution of a task to begin before the current one finishes
- Specify a latency for the completion of functions and loops (one of the reasons for this is to match the FPGA clock frequency).
- Specify a limit to the resources used
- Select the I/O protocol to make sure the final design can be connected to other hardware blocks with the same I/O protocol.

4.3 Analysis of the C Synthesis Results

The first part of the analysis process involves the content of the synthesis reports. We can analyse the performance estimates which involve:

- Timing - the target clock frequency, clock uncertainty and the fastest achievable clock frequency of the design.
- Latency - The latency and initiation interval (II) for the block and any sub-blocks instantiated in the main block. Recall the latency is the number of clock cycles it takes for the program to produce the output, and the initiation interval is the number of clock cycles before new inputs can be processed. A summary of this is included, with greater detail provided in the subsequent section.
- Without pipeline directives (see the section on optimization) the latency is one cycle less than the II.

- Utilization estimates - resources such as the LUTs, FF's, DSP48s used to implement the synthesised design. This also includes the BRAM used, and whether it is single-port or dual-port.

The analysis perspective in Vivado HLS provides a view of the module hierarchy - the resources and latency for each block in the RTL code. This can be observed in more detail in the performance profile for a selected block.

Within the analysis perspective is the schedule viewer tab, which allows for the identification of loop dependencies preventing parallelism, timing violations and data dependencies. One can filter the operations by type or by cluster.

4.4 HLS Directives

Directives applied to an object or scope will apply it to all the sub-objects within. For example, applying a directive to an interface applies it to the top-level function of the C program, as this is the scope containing the interface. Applying a directive to a loop applies it to all objects contained within the loop.

Directives can be stored as a Tcl command within directives.tcl, or within the source code as a pragma. Do the former for transient directives and the latter for likely permanent directives[10], for all solutions. Pragmas must be validated during C synthesis and will output errors if configured incorrectly.

`#pragma` commands for directives include:

- `#pragma HLS PIPELINE II=1` to get Vivado to try to reduce the initiation interval to 1 clock cycle by initiating process during other processes.
- `#pragma HLS unroll [factor = N]` to try to run N iterations of the loop in parallel (omitting the factor = N automatically runs all iterations in parallel) which results in an increase in resources used, but a higher system performance and more throughput.
- `#pragma HLS DATAFLOW` to enable task-level pipelining, allowing functions and loops to overlap in their operation, increasing throughput.
- `#pragma HLS allocation instances=<list>` (nextline) `limit=<value> <type>` specifies the instance restrictions to limit resource allocation in the implemented kernel.

with many more available at `https://www.xilinx.com/html_docs/xilinx2019_1/sdaccel_doc/hls-pragmas-okr1504034364623.html#zof1504034359187`.

4.5 Arbitrary Precision Types

The different numerical data types are as follows:

- char - 8 bit

[10] Such as TRIPCOUNT and INTERFACE

- short - 16 bit
- int - 32 bit
- long long - 64 bit
- float - 32 bit
- double - 64 bit
- int16_t and int32_t - 16 and 32 bit exact width integer types

Creating hardware means that accurate bit-widths are required. Using standard C data types for hardware design results in unnecessary hardware costs. Operations can use more LUTs and registers than needed for the required accuracy, causing delays that may exceed a given clock cycle, increasing the overall latency. Smaller bit widths (e.g using 17 bit data types for 17 bit multiplication rather than the standard C 32 bit data type) result in hardware operators which are smaller and faster. More logic can then fit in the FPGA and can operate at a higher clock frequency.

To implement arbitrary ap_[u]int precision types in C++, include header file ap_int.h in the project source file. Change the bit types to ap_int⟨N⟩ or ap_uint⟨N⟩, where N is a bit-size ranging from 1 to 1024. You can set this limit higher by `#define AP_INT_MAX_W N`, but setting this too high can cause slow software compile and run times.

4.6 Interface Synthesis

Vivado HLS creates three types of ports on the RTL design:

1. Clock and Reset ports: ap_clk, ap_rst
2. Block level interface protocols: ap_start, ap_done, ap_ready and ap_idle.

   These are the signals that control the block independently of any port-level I/O protocols. ap_start dictates when the block can start processing data, ap_ready dictates whether it can accept new inputs, and you can guess what ap_idle and ap_done are responsible for.
3. Port level interface protocols: created for each argument in the top-level function and the function return.

   The I/O protocol depends on the type of C argument and on the switch default option. Port level IO protocols sequence the data in and out of the block once operation has been started by the above protocols. If there is no I/O protocol associated with the block output port, it is difficult to know when to read the data. Therefore it is always a good idea to use one on an output.

The DATA_PACK optimisation directive can pack all the elements of a struct into a single wide vector, allowing all the members of the struct to be read and written to simultaneously. This overrides the default process for structs, where they are decomposed into their member elements and ports are implemented separately for each element. More data can be consequently accessed in a given clock cycle.

When this is the case, Vivado HLS can automatically unroll the loops consuming this data, improving the throughput. This can be controlled by the config_unroll command and the tripcount_threshold option. The syntax is config_unroll -tripcount_threshold <N> where N is the minimum number of loops where the loop is not automatically unrolled.

If a struct contains arrays, those arrays can be optimised using the ARRAY_PARTITION directive to partition the array or the ARRAY_RESHAPE directive to partition the array and also re-combine the partitioned elements into a wider array. This can greatly improve performance. You cannot use DATA_PACK and the the array directives in tandem; they are mutually exclusive.

5 Pedestal Subtraction Algorithm

The pedestal subtraction algorithm is the first sub-block in the TPG core, fed 64 sample words from the Header-Stripper/Combiner, shown in figure 2. The first 6 words (16 bit data types) are stripped from the incoming samples and sent to the pedsub block, where the 'background noise' of the ADC values is removed. The algorithm is implemented as follows:

- The algorithm works by scanning each sample within the packet, subtracting the pedestal from each ADC value and adjusting the pedestal and accumulator variables according to the size of each ADC value. The firmware runs a AXI4 interface between blocks , which operates through a series of boolean signals mentioned in section 1.2. tkeep and treset are signals that are not mentioned previously, the former of which goes high when the incoming data should be processed, and the latter goes high when the entire process must be reset and restarted. The algorithm only runs when tready is low and tvalid and tkeep are high. tlast and tuser go high at the end of a packet. treset goes high when an error in the FELIX interface occurs.

- The user inputs an estimate of the median background value to subtract from the first few samples, as a temporary buffer before the pedestal value is changed by the algorithm.

- The algorithm also starts with an associated accumulator variable, initially equal to zero, that directly changes the pedestal value once it reaches ± 10. How does this accumulator variable change?

- The accumulator changes by ± 1 depending on the boolean (ADCvalue >

`pedestal`). If this boolean is true, the accumulator increases by one, and false means a decrease by one.

- The next condition is: if before a scan the accumulator is +10, the pedestal increases by one, and the accumulator is reset to zero. Then the next ADC value is subtracted by this new increased pedestal. The opposite is the case for an accumulator of −10; the pedestal decreases by one and the accumulator resets.

- At the end of each packet, the current value for the channel is stored and then retrieved from a state save/restore mechanism, for the next packet entering the channel.

Eventually, with a large enough flux of packets, one would obtain a pedestal value appropriate for that input channel, giving more useful ADC values.

5.1 Pedestal Subtraction Testbench

The testbench of any design is one of the most important parts of any HLS project, and usually requires more time to complete than the source algorithm. The main purpose of the testbench is to validate the design when editing the top function so that it can C-simulate within the HLS compiler, synthesize and RTL/cosimulate to then be exported into an IP block. See section 4.1 for more detail on the firmware design flow.

The first testbench design generated an array of random ADC integer values within a reasonable range, and found the ideal pedestal value for that range. Subsequently, the testbench function scanned those ADC values with the subtraction algorithm and validated that the pedestal value converged to the expected value, or was equal to that value after a certain number of packets. This was acceptable to test the first iteration but it did not validate the adjusted ADC values output from the algorithm.

The output from a VHDL implementation of the `pedsub` algorithm was used to compare the output data of the HLS algorithm from one wave of packets travelling through 64 channels. This included a reading of the AXI4 signals, and a contingency for random raises of treset and tready to see if the data was affected. This testbench also included the state save/restore mechanism to see if the output would be affected by multiple packet waves entering a given channel. The result was as expected, which meant the design was ready for IP format packaging.

5.2 Internalising the State Save/Restore (SSR) Mechanism into the Top Function

Once the algorithm VHDL design was verified by the Vivado HLS software, it had to be developed within the Vivado project manager, where the implementation can be tested within Questasim (more commonly known as Modelsim) to check all signals are being received and output as expected. Unfortunately this

simulation revealed that too much of the state save restore mechanism was being performed within the testbench. It was also discovered that there was no output on the adjusted ADC signal. This was due to multiple signal drivers being responsible for the median being subtracted from the original ADC value, which were instantiated through a flawed logic block. This led to a rollback of the top function block to include memory-allocated arrays that save and overwrite the pedestal and accumulator for each channel, in addition to every adjusted ADC sample.

References

[1] Dune science home page. `https://www.dunescience.org/`, 2020. Accessed 11-01-2021.

[2] Susanne Mertens. Direct neutrino mass experiments. *Journal of Physics: Conference Series*, 718:022013, May 2016.

[3] Peter S Cooper. The masses of the neutrinos. *arXiv preprint arXiv:1605.03159*, 2016.

[4] R. Acciarri, M. A. Acero, M. Adamowski, C. Adams, P. Adamson, S. Adhikari, et al. Long-baseline neutrino facility (lbnf) and deep underground neutrino experiment (dune) conceptual design report volume 1: The lbnf and dune projects, 2016.

[5] Makoto Kobayashi and Toshihide Maskawa. Cp-violation in the renormalizable theory of weak interaction. *Progress of theoretical physics*, 49(2):652–657, 1973.

[6] Pasquale Di Bari. An introduction to leptogenesis and neutrino properties. *Contemporary Physics*, 53(4):315–338, Jul 2012.

[7] Carlo Rubbia. The liquid-argon time projection chamber: a new concept for neutrino detectors. Technical report, 1977.

[8] C Adams, M Alrashed, R An, J Anthony, J Asaadi, A Ashkenazi, M Auger, S Balasubramanian, B Baller, C Barnes, et al. Rejecting cosmic background for exclusive neutrino interaction studies with liquid argon tpcs; a case study with the microboone detector. *arXiv preprint arXiv:1812.05679*, 2018.

[9] The Collaboration, B. Abi, R. Acciarri, Mario Acero, Mark Adamowski, C. Adams, D. Adams, P. Adamson, M. Adinolfi, Zubair Ahmad, C. Albright, T. Alion, J. Anderson, K. Anderson, Costas Andreopoulos, M. Andrews, R. Andrews, J.C Anjos, A. Ankowski, and R. Zwaska. The single-phase protodune technical design report. 06 2017.

[10] Andrea Borga, Eric Church, Frank Filthaut, Enrico Gamberini, René Habraken, Giovanna Lehmann Miotto, Thijs Miedema, Frans Schreuder, Jörn Schumacher, Roland Sipos, et al. Felix based readout of the single-phase protodune detector. In *EPJ Web of Conferences*, volume 214, page 01013. EDP Sciences, 2019.

[11] Babak Abi, Giles Barr, Justo Martin-Albo, and Farrukh Azfar. Dune's daq architecture; envisaging the alternatives, Aug 2017.

[12] D Adams, M Bass, M Bishai, C Bromberg, J Calcutt, H Chen, J Fried, I Furic, S Gao, D Gastler, et al. The protodune-sp lartpc electronics production, commissioning, and performance. *Journal of Instrumentation*, 15(06):P06017, 2020.

[13] Konstantinos Manolopoulos. Processing framework & hit finder algorithm, Jul 2020.

[14] J Anderson, K Bauer, A Borga, H Boterenbrood, H Chen, K Chen, G Drake, M Dönszelmann, D Francis, D Guest, et al. Felix: a pcie based high-throughput approach for interfacing front-end and trigger electronics in the atlas upgrade framework. *Journal of Instrumentation*, 11(12):C12023, 2016.

[15] Benjamin Aimard, Ch Alt, J Asaadi, M Auger, V Aushev, D Autiero, MM Badoi, A Balaceanu, G Balik, L Balleyguier, et al. A 4 tonne demonstrator for large-scale dual-phase liquid argon time projection chambers. *Journal of Instrumentation*, 13(11):P11003, 2018.